



Program Studi Bioinformatika
FMIPA IPB University

Modul Praktikum Sistem Basis Data Bioinformatika

Pertemuan 6 Pembersihan dan Integrasi Data Biologis

Prof. Dr. Eng. Wisnu Ananta Kusuma, S.T., M.T.
Dr. Luthfan Irfana, S.Si., M.Si.
Lailan Sahrina Hasibuan, S.Kom., M.Kom.
Gilland Fausta Putra Achyar, S.Mat., M.Kom.
Said Thaufik Rizaldi, S.Kom., M.Kom.
Hakim Ilyas Azhar
Tsabitha Naylasafa Aurora

Daftar Isi

1	Pendahuluan	1
2	Memahami Sumber dan Aturan Data	1
	2.1 Konteks, Berkas, dan Satuan Baris	1
	2.2 Aturan Pengolahan yang Disepakati	2
	2.3 Tugas Prapraktikum	3
3	Menyiapkan Schema dan Mengimpor Data	3
	3.1 Membuka Database dan Membuat Tabel	3
	3.2 Mengimpor Tiga Berkas CSV	4
4	Audit Data Mentah Sebelum Pembersihan	5
	4.1 Memeriksa Jumlah Baris Awal	5
	4.2 Menemukan Masalah Identifier	6
	4.3 Menguji Penggabungan Langsung	6
5	Standardisasi Format dan Nilai Kosong	7
	5.1 Membuat View Standardisasi	7
	5.2 Memeriksa Hasil Standardisasi	8
6	Duplikasi dan Konflik	9
	6.1 Menemukan Duplikat Identik	10
	6.2 Membentuk Data Dedup dan Menemukan Konflik	10
	6.3 Membuat View Bersih	11
7	Pemetaan Identifier Antarsumber	12
	7.1 Meringkas dan Memberi Status Mapping	12
	7.2 Membuat Fitness Terpetakan dan Memeriksa Target Berulang	13
8	Integrasi Data	15
	8.1 Membuat dan Memeriksa Hasil Integrasi	15
	8.2 Memeriksa Fitness di Luar Cakupan RNA	16
9	Validasi dan Ekspor	17
	9.1 Menelusuri Rekaman yang Ditahan	18
	9.2 Mengekspor Hasil	18
10	Latihan Mandiri dan Refleksi	19
11	Mengatasi Kendala	20
12	Lembar Kerja Praktikum	20
	12.1 Berkas yang Dikumpulkan	21
	12.2 Ketentuan Teknis Laporan	21
	12.3 Aspek Penilaian	22

Tujuan Praktikum

1. Mengidentifikasi nilai kosong, perbedaan format, duplikat identik, dan rekaman yang bertentangan.
2. Menerapkan aturan pembersihan tanpa menimpa data mentah.
3. Menggunakan tabel pemetaan untuk membedakan pasangan *identifier* yang unik, tanpa mapping, dan ambigu.
4. Memvalidasi hasil integrasi serta menjelaskan penyebab nilai NULL dan perubahan jumlah baris.

1 Pendahuluan

Pada Pertemuan 5, mahasiswa menggunakan JOIN untuk menghubungkan data RNA-seq, skor mutagenesis, dan anotasi biologis berdasarkan *identifier*. Keberhasilan *query* belum menjamin bahwa pasangan yang terbentuk sudah benar. Spasi tambahan dapat menggagalkan pencocokan, sedangkan duplikasi dan pemetaan *identifier* yang tidak unik dapat memperbanyak baris hasil.

Pertemuan 6 melanjutkan pekerjaan tersebut dari sisi kualitas data. Pertanyaan utamanya adalah bagaimana membersihkan dan memadankan dua sumber data biologis agar hasil integrasinya konsisten dan dapat ditelusuri kembali ke rekaman sumber. Praktikum ini memenuhi cakupan Sub-CPMK6 dalam RPS, yaitu pembersihan dan integrasi data biologi molekuler pada skenario basis data.

Data numerik dalam latihan berasal dari subset BioSQLite, sedangkan kode LAB, tabel pemetaan, dan gangguan kualitas merupakan simulasi pembelajaran. Setiap keputusan pembersihan harus disertai bukti, sementara data mentah tetap dipertahankan agar hasil dapat diperiksa kembali. Hasil latihan tidak digunakan untuk menarik kesimpulan biologis tentang gen tertentu.

2 Memahami Sumber dan Aturan Data

Sebelum menilai kualitas data, tentukan terlebih dahulu objek yang direkam, arti setiap kolom, dan satuan setiap baris. Dua sumber dalam latihan ini memuat data yang berbeda, tetapi akan dihubungkan pada tingkat *identifier* gen. Pemahaman terhadap satuan baris menjadi dasar untuk membedakan duplikasi, konflik, dan hubungan yang sah.

2.1 Konteks, Berkas, dan Satuan Baris

BioSQLite yang digunakan pada Pertemuan 5 memuat data *Plasmodium falciparum*. Pada praktikum ini, RNA-seq diwakili oleh *raw count*, yaitu jumlah pembacaan yang ditetapkan ke suatu *identifier* dalam pemrosesan sumber. *wt_1* dan *ko_1* merupakan dua kolom sampel, sedangkan skor *fitness* mutagenesis berasal dari sumber lain. Praktikum ini mengelola kualitas data tersebut tanpa menilai perubahan ekspresi, esensialitas, atau signifikansi biologis.

Subset dipilih dari 210 *identifier* pertama yang tersedia pada kedua berkas asal setelah diurutkan secara leksikografis. Sumber RNA memakai urutan 1–200, sedangkan sumber *fitness* memakai urutan 1–190 dan 201–210. Susunan ini dibuat untuk latihan cakupan sumber, bukan sebagai sampel yang mewakili seluruh genom.

Kode LAB0001 dan seterusnya merupakan kode lokal simulasi, bukan *accession* resmi atau konversi biologis yang telah diverifikasi. Tabel pemetaan menyimpan pasangan kandidat antara kode lokal dan *identifier* gen; satu kode sengaja memiliki dua kandidat agar ambiguitas dapat diperiksa.

Asal data dan modifikasi paket dirinci dalam `ASAL_DATA.md`.

Data awal praktikum terdiri atas tiga berkas CSV dengan satuan baris yang berbeda. Karena itu, jumlah baris tidak boleh langsung disamakan dengan jumlah gen biologis unik.

Tabel 1. Berkas sumber Praktikum 06 dan satuan barisnya.

Berkas	Isi dan satuan baris
<code>rna_raw.csv</code>	202 rekaman sumber RNA. Satu rekaman dicatat sebagai satu <i>identifier</i> gen dan dua <i>raw count</i> sebelum kualitasnya diperiksa. Data memuat duplikat dan <i>identifier</i> kosong untuk keperluan audit.
<code>fitness_raw.csv</code>	202 rekaman sumber <i>fitness</i> . Satu rekaman memuat satu kode lokal dan satu skor, termasuk duplikat, konflik, dan penanda kosong.
<code>id_mapping.csv</code>	199 pasangan kode lokal- <i>identifier</i> gen. Satu baris adalah satu pasangan kandidat, bukan satu kode lokal unik.

Kolom `row_id` adalah nomor rekaman impor yang digunakan untuk penelusuran. Dua salinan data dapat memiliki `row_id` berbeda sehingga kolom ini tidak digunakan untuk menentukan apakah isi biologisnya identik. Statusnya sebagai *primary key* hanya mencegah nomor rekaman yang sama dimasukkan lebih dari sekali.

Tabel 2. Kolom penting pada data latihan.

Kolom	Tipe dan arti
<code>gene_id_raw / local_id_raw</code>	Teks <i>identifier</i> asli sebelum standardisasi.
<code>wt_1 / ko_1</code>	Bilangan bulat <i>raw count</i> ; nilai kosong tetap NULL, bukan nol.
<code>fitness_raw</code>	Teks skor asli. Sumber memakai koma sebagai desimal; NA, teks kosong, atau spasi saja berarti nilai tidak tersedia dalam data latihan.
<code>local_id / gene_id</code> pada <i>mapping</i>	Pasangan kode lokal simulasi dan <i>identifier</i> tujuan.
<code>mapping_source / mapping_version</code>	Asal dan versi <i>mapping</i> simulasi agar keputusan dapat ditelusuri.

2.2 Aturan Pengolahan yang Disepakati

Aturan berikut digunakan agar setiap keputusan pembersihan dapat diulang dan diperiksa. Aturan ini berlaku untuk kontrak data latihan, bukan aturan umum untuk semua dataset biologis.

Catatan: Aturan Pengolahan

1. Pertahankan semua tabel mentah. Simpan definisi hasil pengolahan sebagai *view* tanpa menimpa data sumber. Isi *view* dihitung dari data sumber saat *query* dijalankan sehingga *view* bukan salinan data yang bersifat tetap.
2. Hapus hanya spasi biasa di tepi *identifier*. Bagian kode, kapitalisasi, dan *identifier* yang hanya mirip nama tidak diubah tanpa dasar.
3. Ubah penanda kosong pada *fitness* menjadi NULL sesuai kamus data. Koma desimal dikonversi menjadi titik sebelum skor dibaca sebagai angka. Skor negatif tidak otomatis salah.
4. Gabungkan salinan yang identik pada seluruh kolom data relevan. Jika *identifier* yang sama masih memiliki lebih dari satu varian data, tandai sebagai konflik dan tahan dari

integrasi.

5. Terima hanya *mapping* satu kode lokal ke satu target. Sebelum integrasi, periksa juga bahwa setiap target menerima paling banyak satu rekaman *fitness* yang layak.
6. Pertahankan seluruh *identifier* RNA yang bersih. Bedakan tidak adanya pasangan *fitness* yang layak dari pasangan yang skornya kosong.

Pada data penelitian, dua nilai berbeda dapat merupakan pengukuran berulang yang sah apabila tersedia kolom sampel atau kondisi. Dalam latihan ini, setiap sumber didefinisikan memiliki satu rekaman per *identifier* untuk kolom yang disediakan, sehingga dua varian perlu diperiksa dan tidak dipilih secara sewenang-wenang.

2.3 Tugas Prapraktikum

Setelah membaca Bagian 1–2, jawab pertanyaan berikut secara individual sebelum praktikum.

Tugas Prapraktikum

1. Jelaskan mengapa nilai NULL berbeda dari nol.
2. Jelaskan kapan *identifier* yang berulang dapat menjadi relasi yang sah.
3. Jelaskan mengapa dua kode berbeda belum tentu menunjuk objek biologis yang berbeda.

Jawaban ditulis tangan secara singkat menggunakan kalimat sendiri pada selembar kertas utuh dan dikumpulkan pada awal praktikum. Tugas prapraktikum ini tidak dicantumkan dalam LKP.

3 Menyiapkan Schema dan Mengimpor Data

Schema merupakan ruang nama untuk mengelompokkan objek dalam database. Praktikum ini menempatkan data baru pada schema **praktikum06** agar mudah dikenali tanpa mengubah tabel pada schema **public**. Ekstrak folder mahasiswa ke lokasi lokal yang mudah ditemukan dan jangan menyimpan ulang CSV dengan Excel sebelum impor karena format teks dapat berubah.

3.1 Membuka Database dan Membuat Tabel

Ikuti langkah berikut untuk menyiapkan ruang kerja.

1. Buka pgAdmin, hubungkan server PostgreSQL, lalu pilih database hasil *restore* yang Anda buat pada Pertemuan 5. Database tersebut harus memuat schema **public** dari praktikum sebelumnya.
2. Klik kanan database tersebut, pilih **Query Tool**, lalu jalankan pemeriksaan berikut.

```
SELECT current_database() AS database_aktif;
```

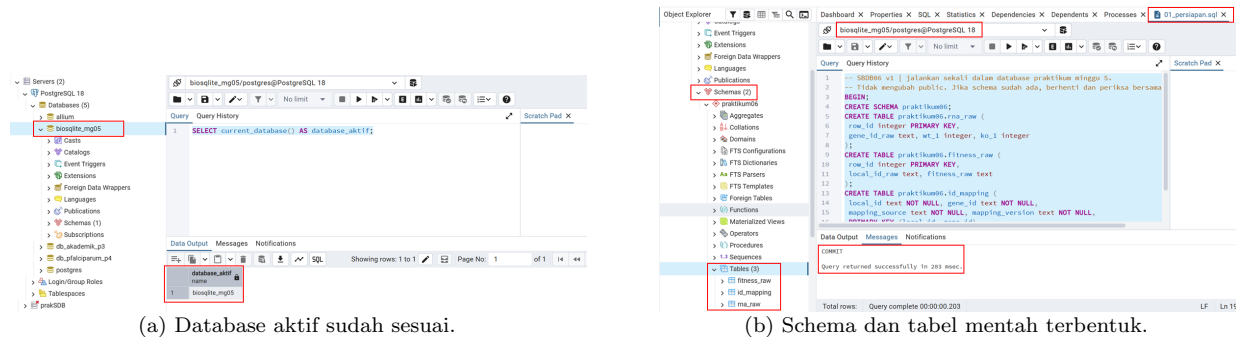
Catat nama database aktif pada lembar kerja. Jika namanya tidak sesuai, buka kembali **Query Tool** dari database yang benar.

3. Buka **01.persiapan.sql**, baca komentarnya, lalu jalankan seluruh skrip satu kali. Skrip tersebut membuat schema **praktikum06**, dua tabel data mentah, dan satu tabel pemetaan dalam satu transaksi.
4. Klik kanan **Schemas** pada panel Browser, pilih **Refresh**, lalu buka **praktikum06** → **Tables**.

5. Pastikan `rna_raw`, `fitness_raw`, dan `id_mapping` terlihat sebelum memulai impor.

Jika schema `praktikum06` sudah ada, jangan menghapusnya atau menjalankan ulang skrip. Periksa pekerjaan yang sudah tersedia.

Pemeriksaan database aktif dan hasil pembuatan schema ditunjukkan pada Gambar 1.



Gambar 1. Pemeriksaan database aktif dan pembentukan schema `praktikum06`.

3.2 Mengimpor Tiga Berkas CSV

Tabel mentah sudah dibuat, tetapi belum berisi data. Masukkan setiap CSV ke tabel yang sesuai tanpa melakukan pembersihan pada tahap impor. Pengaturan impor harus mempertahankan spasi, teks NA, dan sel kosong agar masalah asli masih dapat diperiksa pada tahap audit.

Impor dilakukan melalui **Import/Export Data** pada pgAdmin. Untuk setiap berkas, gunakan pasangan sumber dan tabel tujuan pada Tabel 3.

Tabel 3. Pasangan berkas CSV dan tabel tujuan.

Berkas sumber	Tabel tujuan	Urutan kolom
<code>data/rna_raw.csv</code>	<code>praktikum06.rna_raw</code>	<code>row_id</code> , <code>gene_id_raw</code> , <code>wt_1</code> , <code>ko_1</code>
<code>data/fitness_raw.csv</code>	<code>praktikum06.fitness_raw</code>	<code>row_id</code> , <code>local_id_raw</code> , <code>fitness_raw</code>
<code>data/id_mapping.csv</code>	<code>praktikum06.id_mapping</code>	<code>local_id</code> , <code>gene_id</code> , <code>mapping_source</code> , <code>mapping_version</code>

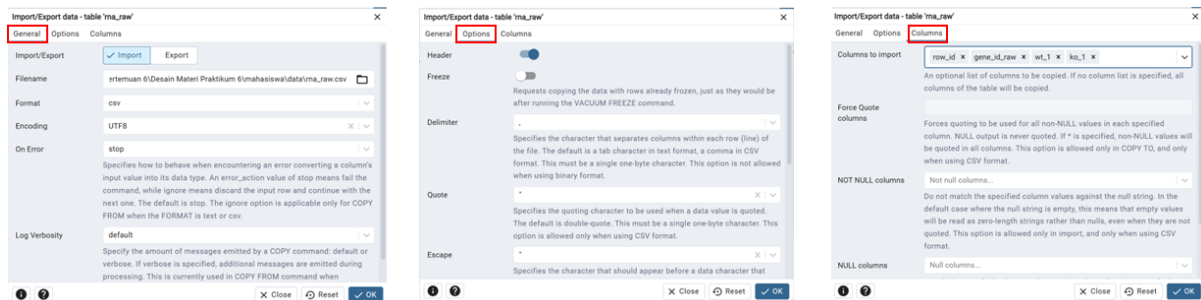
Lakukan langkah berikut untuk setiap tabel.

1. Klik kanan tabel tujuan, pilih **Import/Export Data**, lalu pilih mode **Import**.
2. Pada **Filename**, pilih CSV yang sesuai. Gunakan **Format csv** dan **Encoding UTF8**.
3. Aktifkan **Header**. Tetapkan **Delimiter** sebagai koma (,), **Quote** sebagai tanda petik ganda ("), dan **Escape** sebagai tanda petik ganda (").
4. Biarkan **NULL String** kosong. Jangan aktifkan **Force Not Null** atau **Force Null**. Jika tersedia pilihan **On Error**, gunakan mode yang menghentikan proses ketika ada galat.
5. Periksa tab **Columns** dan pastikan urutan kolom sesuai Tabel 3. Jalankan impor satu kali, lalu baca status dan pesan proses sampai selesai.

Ulangi langkah tersebut untuk setiap pasangan pada Tabel 3 dan pastikan proses selesai tanpa galat atau baris yang dilewati. CSV memakai koma sebagai pemisah kolom. Skor desimal berkoma diapit tanda petik ganda sehingga tetap dibaca sebagai satu nilai. Nama tab atau letak pilihan dapat berbeda antaversi pgAdmin; gunakan nilai pengaturan yang sama.

Jika berkas tidak dapat diakses melalui pgAdmin, hubungi asisten. Jangan menggabungkan impor GUI dengan jalur alternatif karena data dapat masuk dua kali.

Contoh pengaturan impor untuk `rna_raw.csv` ditunjukkan pada Gambar 2. Pola pengaturan yang sama digunakan untuk `fitness_raw.csv` dan `id_mapping.csv`; yang berubah hanya berkas sumber, tabel tujuan, dan daftar kolomnya.



Gambar 2. Pengaturan impor CSV pada pgAdmin menggunakan `rna_raw.csv` sebagai contoh.

4 Audit Data Mentah Sebelum Pembersihan

Data sudah dimuat, tetapi belum layak langsung digabungkan. Audit dilakukan untuk memeriksa dan mencatat keadaan awal tanpa mengubah data. Pada tahap ini, kita memastikan jumlah rekaman, menemukan *identifier* kosong atau berspasi, dan menguji apakah pencocokan langsung masuk akal.

Buka `02_langkah_terpandu.sql`, lalu simpan salinan kerja dengan nama `NIM_Nama_Praktikum06.sql`. Sorot satu *query* atau blok yang dibahas sebelum menekan **Execute**; tanpa sorotan, Query Tool dapat menjalankan seluruh isi editor.

4.1 Memeriksa Jumlah Baris Awal

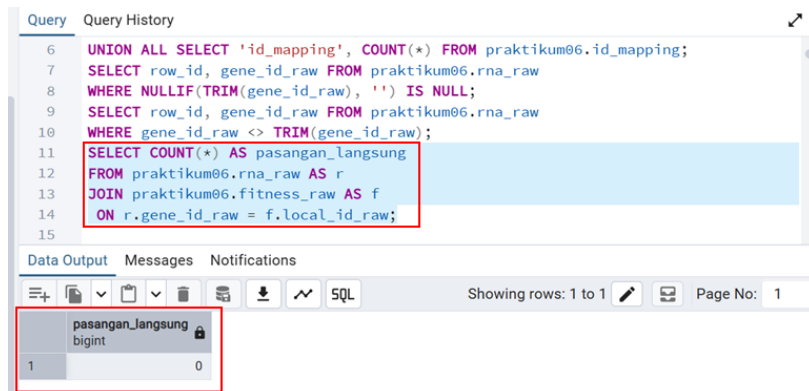
Jalankan query berikut untuk memastikan ketiga tabel latihan telah terisi sesuai berkas sumber.

```
SELECT 'rna_raw' AS tabel, COUNT(*) AS jumlah FROM praktikum06.rna_raw
UNION ALL SELECT 'fitness_raw', COUNT(*) FROM praktikum06.fitness_raw
UNION ALL SELECT 'id_mapping', COUNT(*) FROM praktikum06.id_mapping;
```

Hasil yang diharapkan adalah `rna_raw` berisi 202 baris, `fitness_raw` berisi 202 baris, dan `id_mapping` berisi 199 baris. Angka ini menyatakan jumlah baris yang berhasil dimuat ke tabel latihan, bukan jumlah data yang sudah bersih atau siap diintegrasikan. Hasil pemeriksaan jumlah baris awal ditunjukkan pada Gambar 3.


```
ON r.gene_id_raw = f.local_id_raw;
```

Hasil yang diharapkan adalah `pasangan_langsung = 0`. Hasil tersebut tidak membuktikan bahwa kedua sumber tidak memiliki entitas yang sama. Teks *identifier* pada keduanya menggunakan sistem kode berbeda sehingga diperlukan standardisasi dan tabel *mapping* sebelum integrasi. Hasil penggabungan langsung ditunjukkan pada Gambar 5.



Gambar 5. Penggabungan langsung data mentah belum menghasilkan pasangan.

Tugas Pengamatan

Catat `row_id` untuk *identifier* RNA yang kosong dan yang memiliki spasi di tepi teks. Query spasi juga dapat menangkap rekaman yang hanya berisi spasi, sehingga kategori masalah dapat bertumpang tindih. Jelaskan mengapa jumlah kategori tidak boleh langsung dijumlahkan sebagai jumlah rekaman bermasalah. Catat hasilnya pada tahap Audit awal di `Lembar_Audit.csv`.

5 Standardisasi Format dan Nilai Kosong

Standardisasi menyamakan representasi berdasarkan aturan yang sudah diketahui agar *identifier* dan skor dapat diperiksa secara konsisten. Data mentah tetap dipertahankan, sedangkan hasil standardisasi ditampilkan melalui *view* `rna_norm` dan `fitness_norm` untuk digunakan kembali pada *query* berikutnya.

Beberapa fungsi digunakan berulang pada bagian ini. `TRIM` membuang spasi di tepi teks. `NULLIF(x, '')` menghasilkan `NULL` ketika `x` adalah teks kosong. `CASE` memilih hasil berdasarkan kondisi. `REPLACE` mengganti koma desimal menjadi titik, sedangkan `CAST(... AS numeric)` mengubah teks angka menjadi nilai numerik.

5.1 Membuat View Standardisasi

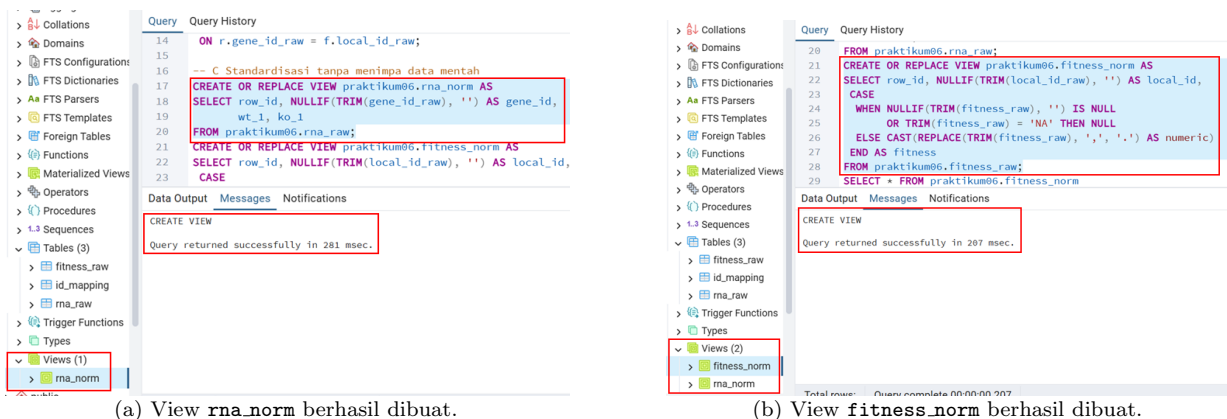
Jalankan *query* berikut dari bagian standardisasi pada berkas `02.langkah_terpandu.sql`. View `rna_norm` menampilkan *identifier* RNA setelah spasi di tepinya dihapus; nilai yang menjadi kosong setelah `TRIM` dicatat sebagai `NULL`.

```
CREATE OR REPLACE VIEW praktikum06.rna_norm AS
SELECT row_id, NULLIF(TRIM(gene_id_raw), '') AS gene_id,
       wt_1, ko_1
FROM praktikum06.rna_raw;
```

View `fitness_norm` membersihkan kode lokal dan mengubah skor *fitness* menjadi nilai numerik. Penanda kosong atau NA diproses lebih dahulu agar tidak ikut dikonversi sebagai angka.

```
CREATE OR REPLACE VIEW praktikum06.fitness_norm AS
SELECT row_id, NULLIF(TRIM(local_id_raw), '') AS local_id,
CASE
WHEN NULLIF(TRIM(fitness_raw), '') IS NULL
OR TRIM(fitness_raw) = 'NA' THEN NULL
ELSE CAST(REPLACE(TRIM(fitness_raw), ',', '.') AS numeric)
END AS fitness
FROM praktikum06.fitness_raw;
```

Setelah kedua view berhasil dibuat, lakukan *refresh* pada schema `praktikum06` dan pastikan `rna_norm` serta `fitness_norm` terlihat pada panel Browser seperti pada Gambar 6.



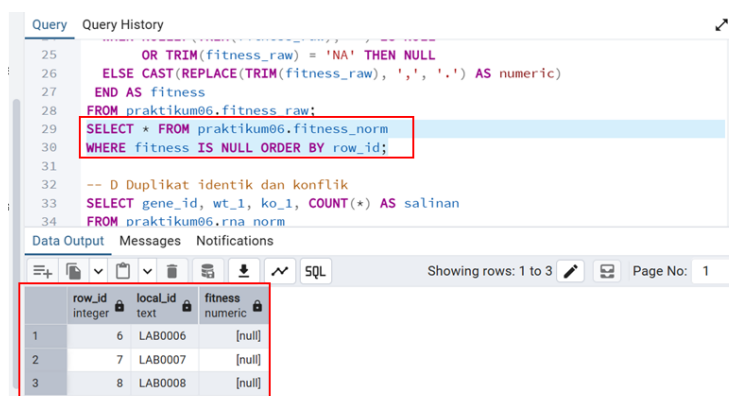
Gambar 6. View hasil standardisasi dibuat tanpa menimpa tabel mentah.

5.2 Memeriksa Hasil Standardisasi

Jalankan *query* berikut untuk memastikan penanda kosong dan NA telah menjadi NULL, bukan nol.

```
SELECT * FROM praktikum06.fitness_norm
WHERE fitness IS NULL ORDER BY row_id;
```

Hasil pemeriksaan nilai *fitness* kosong ditunjukkan pada Gambar 7.



Gambar 7. Tiga rekaman *fitness* memiliki skor NULL setelah standardisasi.

Untuk melengkapi titik pemeriksaan, jalankan dua *query* tambahan berikut. *Query* ini tidak

terdapat dalam 02_langkah_terpandu.sql.

```
SELECT 'rna_norm' AS objek, COUNT(*) AS jumlah
FROM praktikum06.rna_norm
UNION ALL
SELECT 'fitness_norm', COUNT(*)
FROM praktikum06.fitness_norm;
```

```
SELECT *
FROM praktikum06.rna_norm
WHERE gene_id IS NULL OR wt_1 IS NULL
ORDER BY row_id;
```

Contoh hasil pemeriksaan pendukung ditunjukkan pada Gambar 8. Panel pertama memastikan jumlah baris pada kedua view, sedangkan panel kedua memperlihatkan nilai NULL yang masih perlu dicatat pada data RNA terstandarisasi.

objek	jumlah
rna_norm	202
fitness_norm	202

(a) Jumlah baris pada view standardisasi.

row_id	gene_id	wt_1	ko_1
1	PF3D7_0100400	[null]	7
2	202	[null]	12

(b) Nilai NULL pada RNA terstandarisasi.

Gambar 8. Pemeriksaan pendukung jumlah view dan nilai NULL pada RNA terstandarisasi.

Catatan: Titik Pemeriksaan

rna_norm dan **fitness_norm** masing-masing tetap memiliki 202 baris. Tiga rekaman *fitness* memiliki skor NULL. Satu rekaman RNA memiliki *identifier* NULL, dan satu rekaman RNA lain memiliki **wt_1** NULL. Nilai nol pada *count* tetap nol.

Konversi ini dirancang untuk data latihan yang hanya memuat angka, koma desimal, dan penanda kosong yang telah dijelaskan. Jika terdapat teks lain, **CAST** dapat gagal. Jangan langsung mengganti nilai yang gagal menjadi nol atau NULL; periksa data sumber dan tambahkan aturan hanya jika memiliki dasar.

Tugas Pengamatan

Bandingkan nilai asli dan hasil standardisasi pada **row_id** 1, 6, dan 7 di sumber *fitness*. Jelaskan mengapa pembersihan spasi tidak dapat menggantikan tabel *mapping*, dan mengapa skor kosong tidak diisi dengan rerata. Catat hasil dan alasannya pada tahap Standardisasi di **Lembar Audit.csv**.

6 Duplikasi dan Konflik

Pada bagian sebelumnya, nilai sudah dapat dibandingkan secara konsisten. Sekarang kita perlu membedakan salinan identik dari beberapa versi yang bertentangan. Tujuannya adalah memperoleh maksimal satu rekaman layak per identifier sumber, tanpa memilih nilai secara sewenang-wenang.

Duplikat identik ditentukan dari seluruh kolom data yang relevan, bukan dari `row_id`. `DISTINCT` digunakan setelah satuan baris ditetapkan. Setelah salinan identik digabungkan, identifier yang masih muncul pada lebih dari satu baris memiliki beberapa varian dan ditandai sebagai konflik. Query menghitung baris varian, sehingga perbedaan `NULL` versus nilai yang tersedia juga dapat terdeteksi.

6.1 Menemukan Duplikat Identik

Jalankan dua query berikut dari bagian D pada `02_langkah_terpandu.sql`. Query pertama mencari salinan identik pada RNA setelah standardisasi, sedangkan query kedua mencari salinan identik pada skor fitness setelah standardisasi.

```
SELECT gene_id, wt_1, ko_1, COUNT(*) AS salinan
FROM praktikum06.rna_norm
GROUP BY gene_id, wt_1, ko_1 HAVING COUNT(*) > 1;
SELECT local_id, fitness, COUNT(*) AS salinan
FROM praktikum06.fitness_norm
GROUP BY local_id, fitness HAVING COUNT(*) > 1;
```

Hasilnya menunjukkan satu kelompok salinan identik pada masing-masing sumber. Kolom dalam `GROUP BY` menentukan apakah dua rekaman dianggap identik, seperti ditunjukkan pada Gambar 9.

gene_id	wt_1	ko_1	salinan
PF307_0100300	4	1	2

local_id	fitness	salinan
LAB0005	-2.885	2

Gambar 9. Pemeriksaan duplikat identik pada `rna_norm` dan `fitness_norm`.

6.2 Membentuk Data Dedup dan Menemukan Konflik

Setelah duplikat identik dikenali, buat view dedup berikut. `DISTINCT` menggabungkan salinan identik, sedangkan kondisi `IS NOT NULL` menahan rekaman tanpa identifier. Data mentah tetap tidak berubah.

```
CREATE OR REPLACE VIEW praktikum06.rna_dedup AS
SELECT DISTINCT gene_id, wt_1, ko_1 FROM praktikum06.rna_norm
WHERE gene_id IS NOT NULL;
CREATE OR REPLACE VIEW praktikum06.fitness_dedup AS
SELECT DISTINCT local_id, fitness FROM praktikum06.fitness_norm
WHERE local_id IS NOT NULL;
```

Setelah data diringkas, periksa apakah satu identifier masih mempunyai lebih dari satu varian nilai. Pada tahap ini, konflik tidak diselesaikan dengan memilih salah satu nilai. Identifier yang berkonflik dicatat dan ditahan agar tidak diteruskan ke view bersih.

```
CREATE OR REPLACE VIEW praktikum06.rna_conflict AS
SELECT gene_id, COUNT(*) AS n_varian
FROM praktikum06.rna_dedup
GROUP BY gene_id HAVING COUNT(*) > 1;
```

```
CREATE OR REPLACE VIEW praktikum06.fitness_conflict AS
SELECT local_id, COUNT(*) AS n_varian
FROM praktikum06.fitness_dedup
GROUP BY local_id HAVING COUNT(*) > 1;
SELECT * FROM praktikum06.fitness_conflict;
SELECT f.* FROM praktikum06.fitness_norm AS f
JOIN praktikum06.fitness_conflict AS c
ON f.local_id = c.local_id ORDER BY f.row_id;
```

Gambar 10 menunjukkan kode yang berkonflik beserta baris asalnya sehingga perbedaannya dengan salinan identik dapat dilihat.

(a) Identifier fitness yang memiliki konflik.

(b) Detail baris asal yang menyebabkan konflik.

Gambar 10. Konflik fitness ditelusuri sampai ke baris asalnya.

6.3 Membuat View Bersih

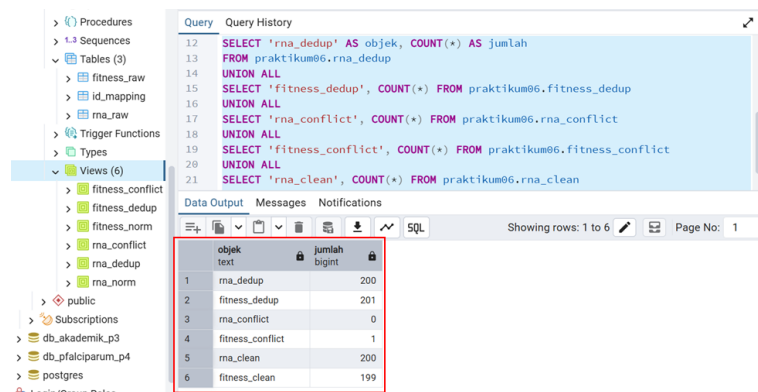
Langkah terakhir pada bagian ini adalah membuat view bersih dengan menahan identifier yang tercatat sebagai konflik. Gunakan LEFT JOIN terhadap view konflik, lalu pertahankan hanya baris yang tidak memiliki pasangan konflik.

```
CREATE OR REPLACE VIEW praktikum06.rna_clean AS
SELECT r.* FROM praktikum06.rna_dedup AS r
LEFT JOIN praktikum06.rna_conflict AS c ON r.gene_id = c.gene_id
WHERE c.gene_id IS NULL;
CREATE OR REPLACE VIEW praktikum06.fitness_clean AS
SELECT f.* FROM praktikum06.fitness_dedup AS f
LEFT JOIN praktikum06.fitness_conflict AS c ON f.local_id = c.local_id
WHERE c.local_id IS NULL;
```

Untuk melengkapi titik pemeriksaan, jalankan query tambahan berikut. Query ini tidak terdapat dalam 02.langkah_terpandu.sql.

```
SELECT 'rna_dedup' AS objek, COUNT(*) AS jumlah
FROM praktikum06.rna_dedup
UNION ALL
SELECT 'fitness_dedup', COUNT(*) FROM praktikum06.fitness_dedup
UNION ALL
SELECT 'rna_conflict', COUNT(*) FROM praktikum06.rna_conflict
UNION ALL
SELECT 'fitness_conflict', COUNT(*) FROM praktikum06.fitness_conflict
UNION ALL
SELECT 'rna_clean', COUNT(*) FROM praktikum06.rna_clean
UNION ALL
SELECT 'fitness_clean', COUNT(*) FROM praktikum06.fitness_clean;
```

Hasil query tambahan ditunjukkan pada Gambar 11.



```
SELECT 'rna_dedup' AS objek, COUNT(*) AS jumlah
FROM praktikum06.rna_dedup
UNION ALL
SELECT 'fitness_dedup', COUNT(*) FROM praktikum06.fitness_dedup
UNION ALL
SELECT 'rna_conflict', COUNT(*) FROM praktikum06.rna_conflict
UNION ALL
SELECT 'fitness_conflict', COUNT(*) FROM praktikum06.fitness_conflict
UNION ALL
SELECT 'rna_clean', COUNT(*) FROM praktikum06.rna_clean
```

objek	jumlah
rna_dedup	200
fitness_dedup	201
rna_conflict	0
fitness_conflict	1
rna_clean	200
fitness_clean	199

Gambar 11. Validasi jumlah baris setelah deduplikasi dan penyaringan konflik.

Catatan: Titik Pemeriksaan

rna_dedup memiliki 200 baris setelah satu salinan identik dan satu identifier kosong dikeluarkan dari hasil layak. **fitness_dedup** memiliki 201 varian. Ada satu kode lokal berkonflik dengan dua varian, sehingga **fitness_clean** menyisakan 199 baris. **rna_conflict** kosong dan **rna_clean** berisi 200 baris.

Nilai 999 pada salah satu rekaman konflik sengaja disisipkan untuk latihan. Rekaman ditahan karena bertentangan dengan kontrak satu nilai per kode, bukan karena kita menetapkan semua skor besar sebagai salah. Kedua varian tetap tersedia pada data mentah dan view konflik. Tidak ada baris data mentah yang dihapus.

Tugas Pengamatan

Temukan dua **row_id** yang mendasari konflik LAB0010. Jelaskan mengapa memilih nilai maksimum, memilih baris pertama, atau merata-ratakannya belum dapat dibenarkan. Sebutkan informasi tambahan yang perlu diminta kepada penyedia data. Catat bukti duplikasi dan konflik serta alasan penahanannya pada tahap Duplikasi dan Konflik di **Lembar_Audit.csv**.

7 Pemetaan Identifier Antarsumber

Pemetaan identifier menyatakan hubungan antara kode pada dua sistem. Berbeda dari pembersihan spasi, proses ini membutuhkan referensi pasangan. Pada latihan ini, **id_mapping** merupakan referensi simulasi yang sudah disediakan. Tujuan bagian ini adalah memisahkan mapping unik, mapping yang tidak tersedia, dan mapping dengan beberapa kandidat.

Satu kode lokal dengan dua target disebut ambigu dalam kontrak latihan ini karena kode tersebut seharusnya menunjuk satu identifier gen. Pada sistem lain, hubungan satu-ke-banyak dapat sah, misalnya satu gen memiliki beberapa transkrip. Status mapping harus ditentukan dari jenis entitas dan tujuan integrasi, bukan dari jumlah pasangan saja.

7.1 Meringkas dan Memberi Status Mapping

Jalankan query berikut dari bagian E pada **02.langkah_terpandu.sql**. View **mapping_summary** menghitung jumlah target berbeda untuk setiap **local_id**. View **mapping_status** kemudian memberi status **tanpa_mapping**, **unik**, atau **ambigu** untuk setiap kode fitness yang sudah bersih.

```
CREATE OR REPLACE VIEW praktikum06.mapping_summary AS
SELECT local_id, COUNT(DISTINCT gene_id) AS n_target
FROM praktikum06.id_mapping GROUP BY local_id;
CREATE OR REPLACE VIEW praktikum06.mapping_status AS
SELECT f.local_id, f.fitness,
CASE WHEN m.local_id IS NULL THEN 'tanpa_mapping'
      WHEN m.n_target = 1 THEN 'unik'
      ELSE 'ambigu' END AS status_mapping
FROM praktikum06.fitness_clean AS f
LEFT JOIN praktikum06.mapping_summary AS m
ON f.local_id = m.local_id;
```

Setelah kedua view dibuat, jalankan query ringkasan berikut untuk memeriksa status setiap kode fitness bersih sebelum tahap integrasi.

```
SELECT status_mapping, COUNT(*) AS jumlah
FROM praktikum06.mapping_status GROUP BY status_mapping;
```

Ringkasan hasilnya ditunjukkan pada Gambar 12.

status_mapping	jumlah
unik	196
ambigu	1
tanpa_mapping	2

Gambar 12. Status mapping memisahkan kode lokal yang unik, ambigu, dan tanpa mapping.

7.2 Membuat Fitness Terpetakan dan Memeriksa Target Berulang

Hanya mapping berstatus unik yang diteruskan ke `fitness_mapped` agar kasus ambigu dan tanpa mapping tidak diberi target secara sewenang-wenang.

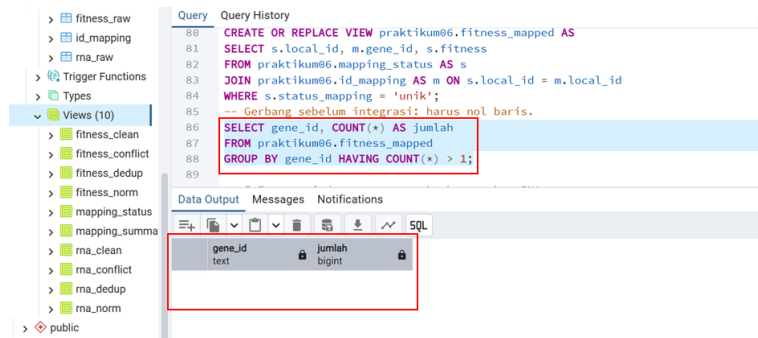
```
CREATE OR REPLACE VIEW praktikum06.fitness_mapped AS
SELECT s.local_id, m.gene_id, s.fitness
FROM praktikum06.mapping_status AS s
JOIN praktikum06.id_mapping AS m ON s.local_id = m.local_id
WHERE s.status_mapping = 'unik';
```

Sebelum menggabungkan fitness dengan RNA, jalankan pemeriksaan target berulang berikut. Hasil yang diharapkan adalah nol baris.

```
-- Gerbang sebelum integrasi: harus nol baris.
SELECT gene_id, COUNT(*) AS jumlah
FROM praktikum06.fitness_mapped
GROUP BY gene_id HAVING COUNT(*) > 1;
```

Gerbang ini memeriksa arah sebaliknya: dua kode lokal yang masing-masing memiliki satu target dapat menuju gen yang sama. Mapping unik per kode belum menjamin satu baris per gen. Jika

query menghasilkan baris, hentikan integrasi dan periksa satuan rekamannya bersama asisten. Hasil nol baris ditunjukkan pada Gambar 13.



Gambar 13. Gerbang sebelum integrasi memastikan tidak ada target gen yang berulang.

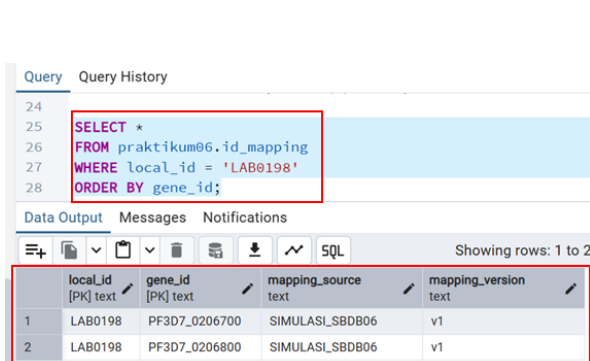
Catatan: Titik Pemeriksaan

Dari 199 kode fitness yang bersih, 196 memiliki mapping unik, satu ambigu, dan dua tanpa mapping. View `fitness_mapped` berisi 196 baris. Pemeriksaan target berulang harus menghasilkan nol baris sebelum melanjutkan.

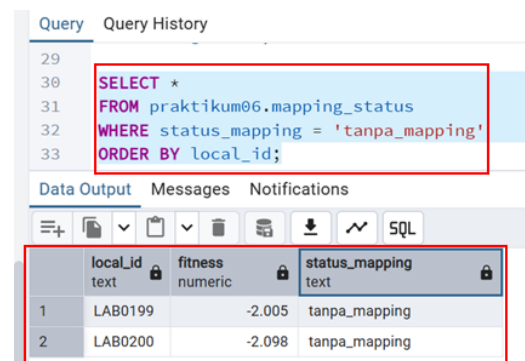
Untuk membantu tugas pengamatan, jalankan dua query tambahan berikut. Keduanya tidak terdapat dalam `02.langkah_terpandu.sql`. Query pertama menampilkan semua kandidat LAB0198, sedangkan query kedua menampilkan kode yang belum memiliki mapping.

```
SELECT *
FROM praktikum06.id_mapping
WHERE local_id = 'LAB0198'
ORDER BY gene_id;

SELECT *
FROM praktikum06.mapping_status
WHERE status_mapping = 'tanpa_mapping'
ORDER BY local_id;
```



(a) Mapping ambigu.



(b) Kode tanpa mapping.

Gambar 14. Mapping ambigu dan kode tanpa mapping perlu ditahan dari integrasi otomatis.

Tugas Pengamatan

Tampilkan semua kandidat LAB0198 serta kode berstatus `tanpa_mapping`. Jelaskan mengapa mengambil salah satu kandidat secara acak akan membuat jumlah pasangan terlihat lebih baik tanpa memperbaiki kebenarannya. Catat `mapping_source` dan `mapping_version` yang digunakan, lalu masukkan hasil pengamatan pada tahap Mapping di `Lembar_Audit.csv`.

8 Integrasi Data

Integrasi menggabungkan informasi dari sumber yang berbeda dengan aturan kesesuaian yang telah diperiksa. `rna_clean` dipilih sebagai sisi kiri karena hasil yang diinginkan adalah daftar seluruh identifier RNA beserta skor fitness jika ada pasangan yang layak. `LEFT JOIN` menjaga cakupan tersebut.

`CASE` memberi status yang membedakan tidak adanya pasangan dari adanya pasangan dengan skor kosong. Pemeriksaan menggunakan `local_id` untuk mengenali pasangan, sebab `fitness` dapat bernilai `NULL` meskipun baris pasangannya ada.

8.1 Membuat dan Memeriksa Hasil Integrasi

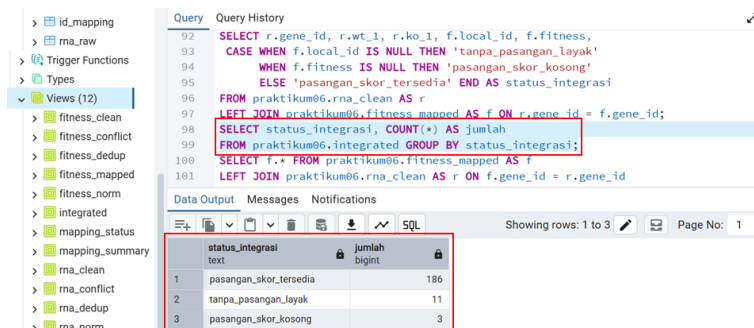
Jalankan query berikut dari bagian F pada `02_langkah_terpandu.sql`. View `integrated` menggabungkan RNA bersih dengan fitness yang sudah dipetakan dan layak diteruskan.

```
CREATE OR REPLACE VIEW praktikum06.integrated AS
SELECT r.gene_id, r.wt_1, r.ko_1, f.local_id, f.fitness,
       CASE WHEN f.local_id IS NULL THEN 'tanpa_pasangan_layak'
            WHEN f.fitness IS NULL THEN 'pasangan_skor_kosong'
            ELSE 'pasangan_skor_tersedia' END AS status_integrasi
FROM praktikum06.rna_clean AS r
LEFT JOIN praktikum06.fitness_mapped AS f ON r.gene_id = f.gene_id;
```

Setelah view dibuat, hitung jumlah baris pada setiap status. Jumlah seluruh status harus kembali ke satuan yang dipertahankan, yaitu baris pada `rna_clean`.

```
SELECT status_integrasi, COUNT(*) AS jumlah
FROM praktikum06.integrated GROUP BY status_integrasi;
```

Ringkasan hasil integrasi ditunjukkan pada Gambar 15.



status_integrasi	jumlah
pasangan_skor_tersedia	186
tanpa_pasangan_layak	11
pasangan_skor_kosong	3

Gambar 15. Status integrasi membedakan pasangan dengan skor tersedia, pasangan dengan skor kosong, dan tanpa pasangan layak.

8.2 Memeriksa Fitness di Luar Cakupan RNA

Pemeriksaan berikut melihat arah sebaliknya: apakah ada rekaman fitness yang sudah terpetakan, tetapi tidak berada pada daftar RNA bersih. Hasil ini tidak mengubah view `integrated`; ia hanya membantu menjelaskan perbedaan cakupan sumber.

```
SELECT f.* FROM praktikum06.fitness_mapped AS f
LEFT JOIN praktikum06.rna_clean AS r ON f.gene_id = r.gene_id
WHERE r.gene_id IS NULL ORDER BY f.gene_id;
```

Tujuh baris pada Gambar 16 tidak masuk ke `integrated` karena arah integrasi mempertahankan seluruh RNA bersih.

Query

Query History

```
97 LEFT JOIN praktikum06.fitness_mapped AS f ON r.gene_id = f.gene_id;
98 SELECT status_integrasi, COUNT(*) AS jumlah
99 FROM praktikum06.integrated GROUP BY status_integrasi;
100 SELECT f.* FROM praktikum06.fitness_mapped AS f
101 LEFT JOIN praktikum06.rna_clean AS r ON f.gene_id = r.gene_id
102 WHERE r.gene_id IS NULL ORDER BY f.gene_id;
103
```

Data Output

Messages

Notifications

≡

📄

📄

🗑️

📄

📄

📄

📄

SQL

Showing rows: 1 to 7

	local_id text	gene_id text	fitness numeric
1	LAB0191	PF3D7_0205900	-3.218
2	LAB0192	PF3D7_0206000	-2.445
3	LAB0193	PF3D7_0206100	-3.284
4	LAB0194	PF3D7_0206200	-2.212
5	LAB0195	PF3D7_0206300	-0.769
6	LAB0196	PF3D7_0206500	-2.766
7	LAB0197	PF3D7_0206600	0

Gambar 16. Tujuh rekaman fitness terpetakan berada di luar cakupan RNA bersih.

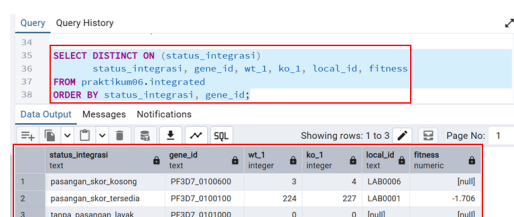
Catatan: Titik Pemeriksaan

Hasil integrasi memiliki 186 baris dengan skor tersedia, 3 baris dengan pasangan tetapi skor kosong, dan 11 baris tanpa pasangan layak. Tujuh rekaman fitness yang sudah terpetakan tidak berada pada daftar RNA bersih.

Label `tanpa_pasangan_layak` adalah status teknis hasil alur ini. Label tersebut tidak menyatakan gen tidak pernah diuji atau tidak memiliki fungsi tertentu. Dalam paket, penyebabnya dapat ditelusuri ke perbedaan cakupan sumber dan penahanan konflik. Demikian pula, skor kosong tidak sama dengan nol.

Untuk membantu tugas pengamatan, jalankan query tambahan berikut untuk menampilkan satu contoh dari setiap status integrasi. Query ini tidak terdapat dalam `02_langkah_terpandu.sql`.

```
SELECT DISTINCT ON (status_integrasi)
    status_integrasi, gene_id, wt_1, ko_1, local_id, fitness
FROM praktikum06.integrated
ORDER BY status_integrasi, gene_id;
```



status_integrasi	gene_id	wt_1	ko_1	local_id	fitness
pasangan_skor_kosong	PF3D7_0100600	3	4	LAB0006	[null]
pasangan_skor_tersedia	PF3D7_0100100	224	227	LAB0001	-1.706
tanpa_pasangan_layak	PF3D7_0101000	0	0	[null]	[null]

Gambar 17. Contoh baris membantu membaca makna setiap status integrasi.

Tugas Pengamatan

Tampilkan satu contoh dari setiap status integrasi. Jelaskan mengapa INNER JOIN akan mengurangi cakupan daftar RNA dan mengapa COUNT(fitness) tidak sama dengan COUNT(local_id). Catat hasil pengamatan pada tahap Integrasi di Lembar Audit.csv.

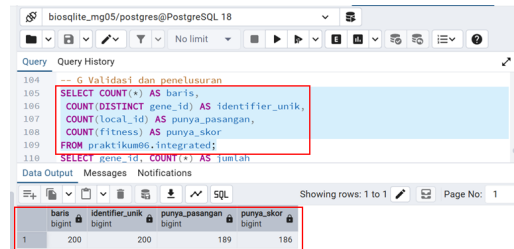
9 Validasi dan Ekspor

Validasi menguji apakah hasil memenuhi aturan yang ditetapkan. Tujuannya bukan hanya mencocokkan angka acuan, tetapi menjelaskan perubahan jumlah baris dari sumber sampai hasil akhir. Periksa satuan hitung: rekaman mentah, varian data, kode lokal, pasangan mapping, dan identifier gen tidak selalu sama.

Jalankan dua query validasi berikut dari bagian G. Query pertama merangkum hasil akhir. Query kedua memastikan tidak ada identifier gen yang muncul lebih dari satu kali pada integrated.

```
SELECT COUNT(*) AS baris,  
COUNT(DISTINCT gene_id) AS identifier_unik,  
COUNT(local_id) AS punya_pasangan,  
COUNT(fitness) AS punya_skor  
FROM praktikum06.integrated;  
SELECT gene_id, COUNT(*) AS jumlah  
FROM praktikum06.integrated GROUP BY gene_id HAVING COUNT(*) > 1;
```

Gambar 18 merangkum seluruh baris hasil, identifier unik, baris dengan pasangan fitness, dan baris dengan skor tersedia.



The screenshot shows a PostgreSQL query editor with a query window and a results window. The query window contains the following SQL code:

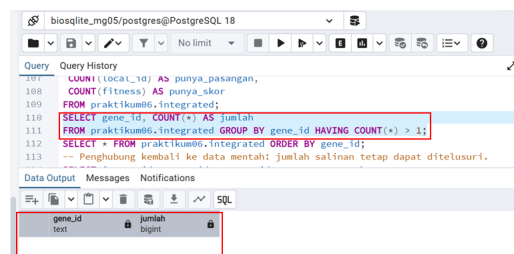
```
-- G Validasi dan penelusuran  
SELECT COUNT(*) AS baris,  
COUNT(DISTINCT gene_id) AS identifier_unik,  
COUNT(local_id) AS punya_pasangan,  
COUNT(fitness) AS punya_skor  
FROM praktikum06.integrated;  
SELECT gene_id, COUNT(*) AS jumlah  
FROM praktikum06.integrated GROUP BY gene_id HAVING COUNT(*) > 1;
```

The results window shows the output of the first query:

baris	identifier_unik	punya_pasangan	punya_skor
1	200	200	189

Gambar 18. Validasi ringkasan memastikan hasil akhir sesuai satuan keluaran.

Pemeriksaan identifier berulang harus menghasilkan nol baris seperti pada Gambar 19. Jika ada baris, hasil akhir tidak lagi memiliki satu baris per identifier gen.



The screenshot shows a PostgreSQL query editor with a query window and a results window. The query window contains the following SQL code:

```
SELECT COUNT(*) AS punya_pasangan,  
COUNT(fitness) AS punya_skor  
FROM praktikum06.integrated;  
SELECT gene_id, COUNT(*) AS jumlah  
FROM praktikum06.integrated GROUP BY gene_id HAVING COUNT(*) > 1;  
SELECT * FROM praktikum06.integrated ORDER BY gene_id;
```

The results window shows the output of the second query:

gene_id	jumlah
text	bigint

Gambar 19. Pemeriksaan identifier berulang menjadi gerbang akhir sebelum ekspor.

Catatan: Titik Pemeriksaan

Hasil akhir adalah 200 baris, 200 identifier unik, 189 baris dengan pasangan fitness, dan 186 skor tersedia. Query identifier berulang menghasilkan nol baris. Gunakan persamaan pemeriksaan berikut: $200 = 186 + 3 + 11$ untuk seluruh daftar RNA; $189 = 186 + 3$ untuk pasangan; dan $196 = 189 + 7$ untuk seluruh fitness yang terpetakan. Ketiga persamaan menguji aspek cakupan yang berbeda.

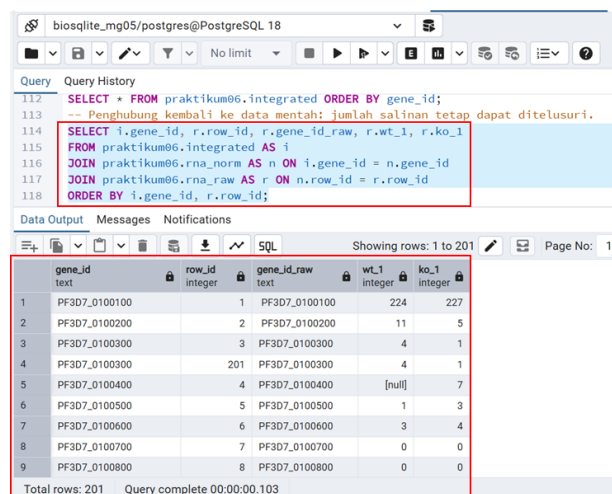
Selanjutnya, jalankan kedua query berikut secara terpisah. Query pertama menampilkan hasil akhir yang akan diekspor, sedangkan query kedua menelusuri hasil tersebut ke baris RNA mentah.

```
SELECT * FROM praktikum06.integrated ORDER BY gene_id;
```

Setelah itu, pilih dan jalankan query penelusuran berikut.

```
-- Penghubung kembali ke data mentah: jumlah salinan tetap dapat ditelusuri.
SELECT i.gene_id, r.row_id, r.gene_id_raw, r.wt_1, r.ko_1
FROM praktikum06.integrated AS i
JOIN praktikum06.rna_norm AS n ON i.gene_id = n.gene_id
JOIN praktikum06.rna_raw AS r ON n.row_id = r.row_id
ORDER BY i.gene_id, r.row_id;
```

Pada data latihan, query penelusuran menghasilkan 201 baris karena satu salinan identik yang telah digabungkan masih dapat ditelusuri ke kedua baris mentahnya. Gambar 20 memperlihatkan kaitan hasil bersih dengan row_id dan nilai asli pada tabel sumber.



gene_id	row_id	gene_id_raw	wt_1	ko_1
PF3D7_0100100	1	PF3D7_0100100	224	227
PF3D7_0100200	2	PF3D7_0100200	11	5
PF3D7_0100300	3	PF3D7_0100300	4	1
PF3D7_0100300	201	PF3D7_0100300	4	1
PF3D7_0100400	4	PF3D7_0100400	[null]	7
PF3D7_0100500	5	PF3D7_0100500	1	3
PF3D7_0100600	6	PF3D7_0100600	3	4
PF3D7_0100700	7	PF3D7_0100700	0	0
PF3D7_0100800	8	PF3D7_0100800	0	0

Gambar 20. Penelusuran raw mempertahankan jejak sumber setelah data dibersihkan.

9.1 Menelusuri Rekaman yang Ditahan

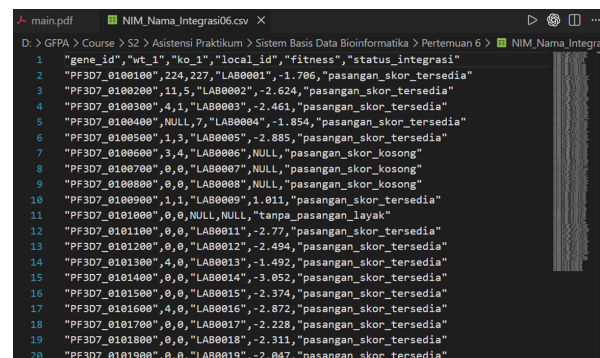
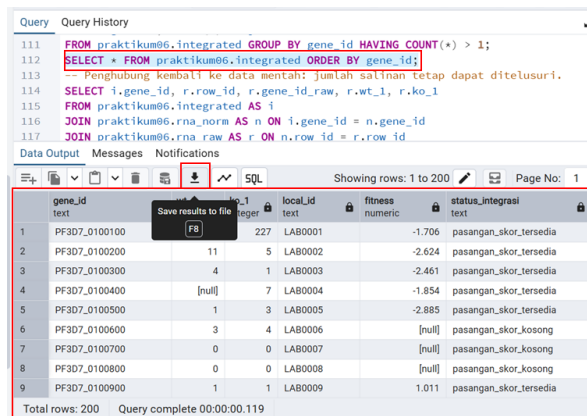
Buat daftar terpisah untuk identifier RNA kosong, kode fitness berkonflik, mapping ambigu, serta kode tanpa mapping. Daftar tersebut adalah bagian hasil praktikum. Catatan masalah harus menyertakan tabel asal, row_id atau kode terkait, alasan ditahan, dan tindakan lanjut yang diperlukan. Catat setiap masalah pada tahap yang sesuai di *Lembar Audit.csv*.

9.2 Mengekspor Hasil

Ikuti langkah berikut untuk menyimpan hasil akhir sebagai berkas CSV.

1. Jalankan hanya query hasil akhir, yaitu `SELECT * FROM praktikum06.integrated` diikuti `ORDER BY gene_id;`, sehingga hasil akhir tampil pada *Data Output*.
2. Gunakan tombol unduh atau *Save Results to File* pada *Data Output*. Simpan sebagai `NIM>Nama_Integrasi06.csv` dan pastikan header kolom ikut tersimpan.
3. Buka berkas dengan editor teks untuk memeriksa header dan beberapa baris. Jangan menganggap tampilan spreadsheet yang memformat ulang kode sebagai isi asli CSV.
4. Simpan skrip kerja dan catatan audit. View bukan snapshot permanen; CSV ini merekam hasil pada saat ekspor.

Letak tombol ekspor dan pemeriksaan CSV ditunjukkan pada Gambar 21.



(a) Menyimpan hasil query ke CSV.

(b) Memeriksa isi CSV tanpa format ulang spreadsheet.

Gambar 21. Ekspor hasil akhir perlu disertai pemeriksaan isi berkas CSV.

Tugas Pengamatan

Isi `Lembar_Audit.csv` dan jelaskan seluruh perubahan jumlah baris. Mengapa angka hasil yang sama saja belum cukup untuk menjamin dua prosedur pembersihan setara?

10 Latihan Mandiri dan Refleksi

Anda telah menjalankan alur lengkap dengan aturan yang disediakan. Latihan berikut menguji apakah Anda memahami alasan keputusan dan dapat menerapkan pola query pada pertanyaan baru. Gunakan `SELECT` agar data mentah tetap sama. Kerjakan secara individual dan simpan query pada berkas `NIM>Nama_Praktikum06.sql`.

Latihan A. Cakupan dua sumber. Gunakan `FULL OUTER JOIN` antara `rna_clean` dan `fitness_mapped`. Hitung jumlah identifier gabungan, identifier yang hanya ada di RNA, dan yang hanya ada di fitness. Jelaskan perbedaan dengan `integrated`.

Latihan B. Diagnosis mapping. Buat query yang menampilkan setiap kode ambigu beserta seluruh kandidat target dan sumber mapping. Tulis alasan mengapa masalah tersebut harus diselesaikan sebelum skor dilekatkan ke suatu gen.

Latihan C. Nilai kosong. Bandingkan `COUNT(*)`, `COUNT(local_id)`, dan `COUNT(fitness)`

pada **integrated**. Jelaskan apa yang dihitung masing-masing dan mengapa mengganti nilai NULL pada **fitness** dengan 0 mengubah makna data.

Latihan D. Jejak sumber. Tampilkan rekaman raw yang menjadi asal dua varian LAB0010. Tulis keputusan sementara serta informasi yang diperlukan untuk menyelesaikan konflik.

Refleksi akhir

Tulis satu atau dua paragraf: bagian mana dari pekerjaan ini dapat diotomatisasi setelah aturannya jelas, dan bagian mana memerlukan informasi biologis atau dokumentasi sumber? Hubungkan jawaban dengan minggu ke-7, ketika data baru dari repositori akan perlu diperiksa sebelum digabungkan dengan database lokal.

11 Mengatasi Kendala

Apabila suatu tahap gagal, periksa penyebabnya sebelum mengulang impor atau mengubah data. Catat query, nama database aktif, pesan lengkap, dan tahap terakhir yang berhasil. Informasi tersebut membantu asisten membedakan masalah sintaks, impor, dan logika.

Tabel 4 merangkum gejala yang sering muncul beserta pemeriksaan awalnya.

Tabel 4. Kendala umum dan tindakan pemeriksaan pada Praktikum 06.

Gejala	Pemeriksaan dan tindakan
<code>schema praktikum06 already exists</code>	Jangan jalankan ulang persiapan atau menghapus schema. Periksa pekerjaan yang sudah ada; gunakan database latihan baru jika perlu bersama asisten.
<code>current transaction is aborted</code>	Jalankan ROLLBACK; setelah membaca galat awal. Perbaiki penyebab sebelum mengulang skrip.
<code>duplicate key saat impor</code>	Periksa apakah berkas telah diimpor atau tabel tujuan salah. Jangan menghapus rekaman untuk melewati galat.
Jumlah baris tidak sesuai	Periksa <i>Header</i> , delimiter, tabel tujuan, status proses, dan apakah impor terulang.
<code>invalid input syntax for type numeric</code>	Periksa nilai <code>fitness_raw</code> dan aturan koma desimal. Jangan mengganti nilai secara massal tanpa diagnosis.
<code>relation does not exist</code>	Periksa database aktif, nama schema, serta urutan pembuatan view. Refresh panel hanya memperbarui tampilan.
Hasil JOIN bertambah	Periksa keunikan target mapping dan satuan baris. Jangan langsung memakai DISTINCT untuk menutupi masalah.
Hasil akhir berbeda setelah data berubah	View dihitung dari data saat query dijalankan. Periksa sumber dan ulangi validasi; simpan CSV untuk hasil pada satu waktu.

12 Lembar Kerja Praktikum

Simpan *query* terpandu dan *query* jawaban Latihan A–D di dalam berkas SQL, lengkap dengan komentar singkat tentang tujuannya. Di dalam LKP, sajikan hasil dan interpretasinya, termasuk

jawaban setiap **Tugas Pengamatan**, Latihan A–D, serta refleksi akhir. Dengan demikian, berkas SQL menunjukkan proses yang dapat dijalankan kembali, sedangkan LKP menjelaskan bukti dan alasan di balik hasil tersebut.

12.1 Berkas yang Dikumpulkan

Berkas yang dikumpulkan pada praktikum ini dirangkum pada Tabel 5.

Tabel 5. Berkas yang dikumpulkan pada Praktikum 06.

Berkas yang dikumpulkan	Isi
NIM>Nama_Praktikum06.sql	Query terpandu dan <i>query</i> jawaban Latihan A–D, disusun berurutan dan dilengkapi komentar singkat tentang tujuan setiap bagian.
NIM>Nama_Integrasi06.csv	200 baris hasil integrasi dan header kolom.
NIM>Nama_Laporan06.pdf	LKP yang memuat audit, bukti hasil, jawaban Tugas Pengamatan dan Latihan A–D, interpretasi, serta refleksi.

Gunakan `Lembar_Audit.csv` sebagai alat bantu kerja, kemudian salin atau rangkum isinya ke dalam LKP. Berkas audit tidak perlu diunggah secara terpisah.

Laporan disusun ringkas, tetapi harus cukup lengkap untuk menunjukkan keputusan pembersihan dan integrasi. Isi minimal laporan adalah sebagai berikut.

1. Identitas dan tujuan singkat; jelaskan hubungan dengan praktikum minggu ke-5.
2. Deskripsi sumber dan arti satu baris; jelaskan bagian yang merupakan simulasi.
3. Tabel audit awal, aturan pembersihan, dan alasan keputusan.
4. Hasil standardisasi, duplikasi, konflik, mapping, integrasi, dan validasi; sertakan jumlah beserta satuannya dan jawaban Tugas Pengamatan yang berkaitan dengan setiap tahap.
5. Jawaban Latihan A–D dan refleksi akhir. Jika menggunakan sumber tambahan, cantumkan hanya referensi yang benar-benar digunakan.

12.2 Ketentuan Teknis Laporan

Susun LKP dengan ketentuan teknis pada Tabel 6.

Tabel 6. Ketentuan teknis laporan Praktikum 06.

Komponen	Ketentuan
Format berkas	LKP dikumpulkan dalam format PDF, skrip dalam format <code>.sql</code> , dan hasil integrasi dalam format <code>.csv</code> .
Margin halaman	Kiri 4 cm; atas, kanan, dan bawah masing-masing 3 cm.
Teks isi	Ukuran huruf 12 pt dengan spasi 1,5.
Judul bagian	Ukuran huruf 14 pt dan dicetak tebal.

Tabel 6 (lanjutan)

Komponen	Ketentuan
Halaman awal	Cantumkan identitas praktikan dan daftar isi sebelum bagian utama LKP.
Gambar dan tangkapan layar	Memiliki nomor dan keterangan di bawah gambar; <i>screenshot</i> harus terbaca serta memperlihatkan <i>query</i> dan hasil yang dibahas. Gunakan tangkapan layar hasil pengerjaan sendiri; gambar pada modul hanya menjadi contoh hasil yang diharapkan.
Tabel	Memiliki nomor dan keterangan di atas tabel; tabel audit mencantumkan jumlah beserta satuannya.
Lampiran data	Tidak perlu menempelkan seluruh baris data; gunakan cuplikan yang relevan dan rujuk berkas <code>NIM_Nama_Integrasi06.csv</code> .

12.3 Aspek Penilaian

Aspek penilaian praktikum dirangkum pada Tabel 7.

Tabel 7. Aspek penilaian Praktikum 06.

Aspek penilaian	Bobot
Pemahaman sumber dan audit awal	15%
Aturan serta penerapan pembersihan	25%
Pemetaan dan integrasi	25%
Validasi dan penelusuran hasil	20%
Latihan mandiri dan komunikasi hasil	15%